CIS 1902: Python

Week 4

Ash Fujiyama

Slides adapted from Jorge Mendez, Arun Kirubarajan, and Tony Liu

Scheduled Topics

- Logistics
- Decorators
- Modules
- Scripting
- Testing



Logistics

- HW2 (NYTimes) released, due in 2 weeks (10/19)
- HW 1 due next week (10/12)
- HW0 Grades released
  - Be sure to check for comments, regardless of your score!

First Class Functions

- Last class, we passed functions as arguments into another function
  - sort(key = lambda x: x.age)

- And looked at functions used as a return value:

```
def make_incrementor(n):
    """returns an incrementor function from the starting value n"""
    # lambdas can reference variables from containing scope
    return lambda x: x + n
```

What does it mean to return a function?

```
start_50 = make_incrementor(50)
print(start_50(1)) # prints 51
```

Generators

- Think about a generator, and the `yield from` keywords

```
def count_to_four():
    yield from [1,2,3,4]


def count_to_four_from_zero():
    yield 0
    yield from count_to_four()
```

- In this example, we've created a generator that modifies an existing generator

Functions and Generators

```
def count_to_four():
    yield from [1,2,3,4]
def count_to_four_from_zero():
    yield 0
    yield from count_to_four()
```

- Which one is the function, and which one is the generator?
  a. `count_to_four()`
  b. `count_to_four`

Functions and Generators

```
def count_to_four():
    yield from [1,2,3,4]
def count_to_four_from_zero():
    yield 0
    yield from count_to_four()
```

- Which one is the function, and which one is the generator?
  a. `count_to_four()  # here we are calling the function, which`
     `# returns a generator`
  b. `count_to_four  # this is the function object`

Decorators

- Definition: A function used to modify a function or class definition
- Remembering our previous generator example (a generator modifying an existing generator)

```
def count_to_four():
    yield from [1,2,3,4]


def count_to_four_from_zero():
    yield 0
    yield from count_to_four()
```

- You can think of decorators as something like this, but with functions instead!

What does decorator() do?

```
def decorator(base_function):
    def wrapper():
        print("running: {}".format(base_function.__name__)
        return base_function()
    return wrapper
```

What does decorator() do?

```
def decorator(base_function):
    def wrapper():
        print("running: {}".format(base_function.__name__)
        return base_function()
    return wrapper

def do_something():
    print("I did something")

decorated_do_something = decorator(do_something)
decorated_do_something()
```

Another way of writing it

```
def decorator(base_function):
    def wrapper():
        print("running: {}".format(base_function.__name__))
        return base_function()
    return wrapper

@decorator
def do_something():
    print("I did something")

do_something()
```

Decorators

- Common use cases:
  - Debugging
  - Efficiency testing (timing how long functions take to run)

- Built-in decorators also exist
  - eg: @staticmethod: used to decorate a static method defined in a class

Scripting

Scripting As We Know It

- .py files
- executed by running `python3 script.py`
- Example: `generator.py` as a basic script
  - we implement functions in the file
  - then, we could run the functions in the top-level environment (under `if __name__ == "__main__"`).

Modules

- when writing a .py file, we are also creating a module **containing functions and class definitions**
  - can be **imported** into other modules
  - access the name of the current module with `__name__`

- may contain executable statements along with definitions
  - these statements are run upon importing the module

- modules define a namespace
  - global variables live within the module
  - access them from outside the module via `module_name.attr`

Importing Modules

- By convention, place import statements at the top

- `import module_name`: access functions via `module_name.func`

Variations of an `import` statement

- **`from module_name import func:`** access functions directly via **`func`**
- **`from module_name import *:`** import all names that don't begin with **`_`**
  - be careful!
- **`import module_name as mn:`** access functions via **`mn.func`**

Example: generator.py as module

Importing Other Modules

- There are numerous standard modules provided by Python that you can import

- https://docs.python.org/3/library/

Scripting in More Detail

- Argparse is a standard library that provides high-level argument parsing
    - Not the only library: sys module handles low level details like script flags
- You can use #! to tell terminal which python interpreter to use
    - Eg: #! usr/bin/python3
    - Need to give file execution permissions before running

Example: generator.py with argument parsing

Testing

Testing

- we all write tests -- little print statements to verify the output, etc.
- but we (usually) end up throwing them away

Testing using assert

- assert statements are a simple way to debug beyond print statements
- Syntax: `assert bool_expression, description`

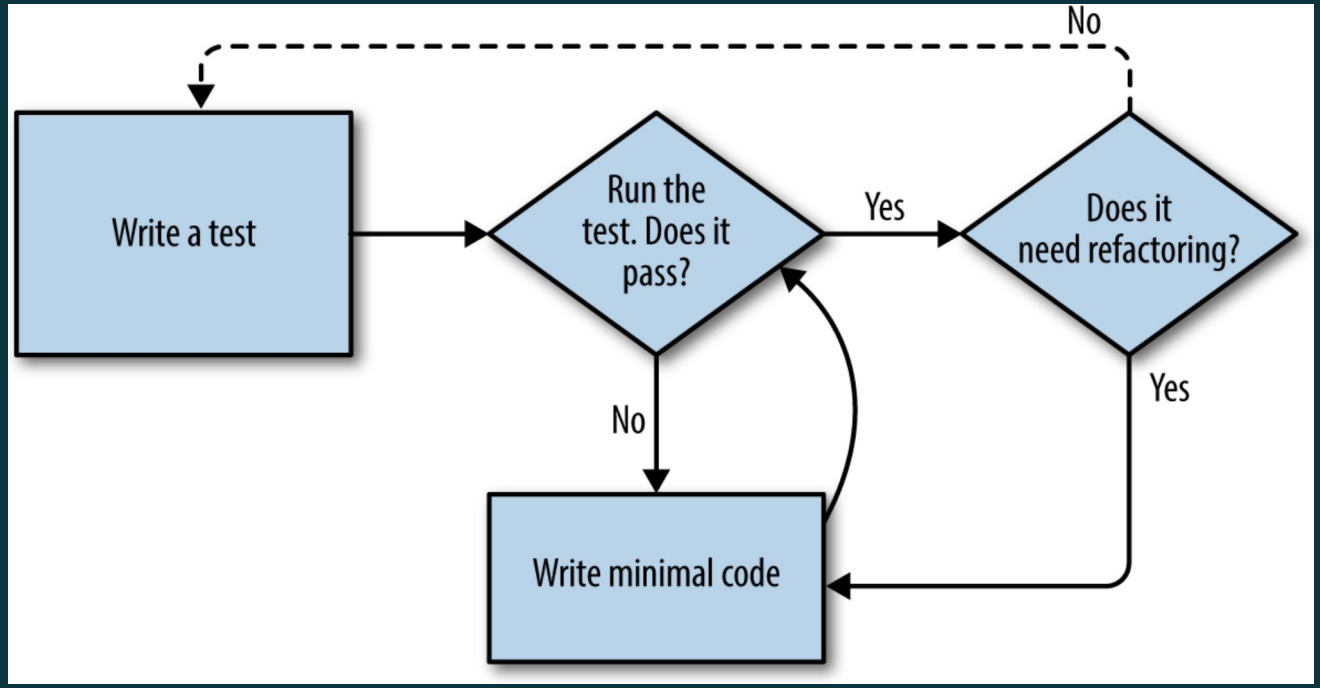
What are unit tests?

- Unit tests check the functionality of specific pieces of code for correct behavior, in isolation
  - In contrast to *integration tests*, which tests how separate components may interact with each other during the integration step

unittest

- `unittest` is a standard library for unit testing in an object-oriented manner
- Subclass the `unittest.TestCase` class to write custom test cases
- Any method defined in the class starting with `test_` is considered a test case
- Numerous `assert...` statements, see documentation for full details
  - Most common is `assertEqual(a, b)`: checks that `a == b`

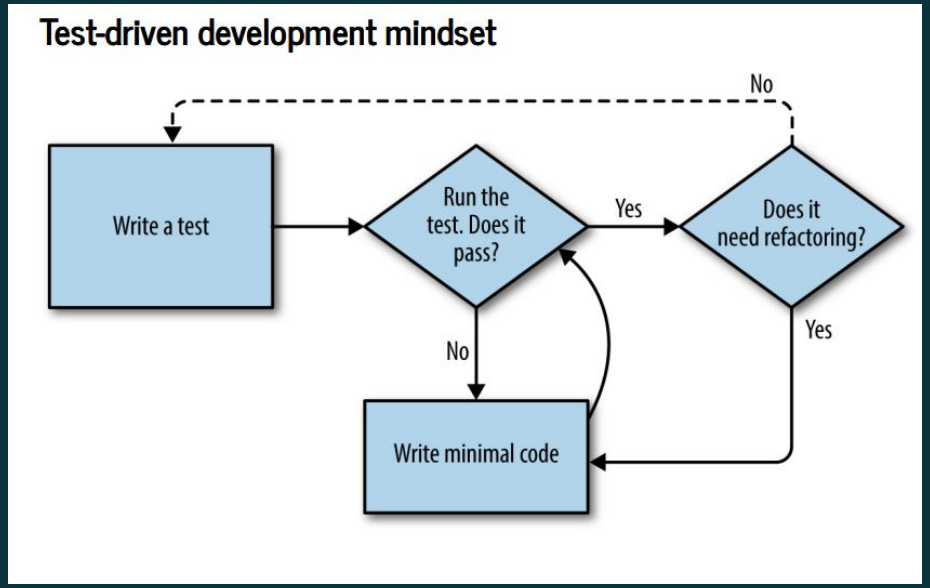
Test-Driven Development Mindset



img credit: Obey the Testing Goat

Test-driven development mindset

- write code to pass tests -> makes coding sessions more directed
- forces you to think about failure points in your code



Obey the Testing Goat

Extended example: Querying Wikipedia

Goal: build a script that provides Wikipedia summaries of topics to the terminal

Topics:

- HTTP
- JSON
- 3rd party modules: installing request

HTTP requests

- HTTP(hypertext transfer protocol) is how client computers and servers exchange information over the internet
  - See the shared 19xx lecture (https://www.cis.upenn.edu/~cis19x/)

- The client (usually your computer's web browser) sends a HTTP request to a specific URL, and the server responds with data, such as the HTML for a webpage

- We can also send and receive requests programmatically in our Python code

Wikipedia summaries

- Wikipedia has an API to extract article summaries
  - https://en.wikipedia.org/api/rest_v1/page/summary/article_name
  - API = “Application Programming Interface”
  - APIs allow 2 separate software programs to connect without requiring knowledge of each other’s implementation

- Sending **`HTTP GET`** requests will return summary data about the article

Parsing data: JSON

- Data from the internet needs to follow particular standards in order to be parsed by machines/code

- JSON is a particularly popular data standard to do this
  - “JavaScript Object Notation”
  - Uses plain text to store data and key-value pairs ← very readable to humans

Parsing data: JSON

- Data from the internet needs to follow particular standards in order to be parsed by machines/code

- JSON is a particularly popular data standard to do this
  - “JavaScript Object Notation”
  - Uses plain text to store data and key-value pairs ← very readable to humans

- JSON concepts map cleanly onto Python
  - JSON objects {} → dictionaries
  - JSON arrays [] → lists
  - JSON values "", 1, 1.0, true → strings, ints, floats, bools

- use the json Python standard library for parsing

Accessing data: JSON

- Read from a file (with .json extension)

```
with open("file.json", "r") as f:
    data = json.load(f)
print(data["name"]
```

- Receive as a response to an HTTP request

```
response = requests.get(wiki_url.format(topic), headers=headers)
data = response.json()
print(response_json["name"])
```

Example: parsing Wikipedia data using json and files

requests: high-level HTTP API

- requests is a 3rd party module that abstracts HTTP response/requests into Python objects

- For reference, compare to lower-level urllib module in the Python standard library

Installing 3rd party modules

- We use pip to install third party modules
  - “PIP Installs Packages”
  - Serves as a Python package installer and manager

- `pip3 install module` or `python3 -m pip install module`

Example: querying the Wikipedia API using requests

Putting everything together

argparse reference

```
# creates an ArgumentParser, automatically provides the -h flag
parser = argparse.ArgumentParser(description="this describes the script")

# add_argument() will create "pos_arg" that can be referenced later from args
parser.add_argument("pos_arg", type=str,
help="this is a required argument parsed as a str")

# passing a name that starts with "-" indicates it is an optional argument
parser.add_argument("-opt_arg", type=int,
help="this is an optional argument parsed as an int")

# arguments can also take "actions", where "store_true" assigns True if the argument is used
parser.add_argument("-bool_flag", action="store_true",
help="this is an optional argument that stores a bool")

# this will parse the arguments passed and store the arguments as attributes of args
args = parser.parse_args()
print("pos_arg is {} with type {}".format(args.pos_arg, type(args.pos_arg)))
print("opt_arg is {} with type {}".format(args.opt_arg, type(args.opt_arg)))
print("bool_flag is {} with type {}".format(args.bool_flag, type(args.bool_flag)))
```

argparse reference: output

```
# sample output when placing the above code in 'demo_parser.py' file
$ python3 demo_parser.py -h
usage: demo_parser.py [-h] [-opt_arg OPT_ARG] [-bool_flag] pos_arg
this describes the script
positional arguments:
 pos_arg this is a required argument, which is parsed as a str
optional arguments:
 -h, --help show this help message and exit
 -opt_arg OPT_ARG this is an optional argument, which is parsed as an int
 -bool_flag this is an optional argument that stores a bool
# sample output when placing the above code in 'demo_parser.py' file
$ python3 demo_parser.py hello -opt_arg 5 -bool_flag
pos_arg is hello with type <class 'str'>
opt_arg is 5 with type <class 'int'>
bool_flag is True with type <class 'bool'>
```

unittest reference

```
# to write a test class, we subclass unittest.TestCase
class MyTests(unittest.TestCase):

# functions that begin with "test_" are considered tests
def test_list_equals(self):
"""Checks whether two lists are equal"""
# ^^^ the docstring will be printed as part of the
# unittest.main() output if -v is specified
expected_list = [1, 2, 3, 4]
actual_list = [1, 2, 3, 4]

# self.assertEqual() tests equality via the '==' operator
self.assertEqual(expected_list, actual_list)

if __name__ == "__main__":
# unittest has its own main function that runs all the test cases
# present within the file
unittest.main()
```

unittest reference: output

```
# sample output when placing the above code in 'mytest.py' file
# the single '.' below indicates one test was run
$ python3 mytest.py
.
----------------------------------------------------------------------
Ran 1 test in 0.000s
OK
# -v indicates verbose output, which prints the test name and docstring
$ python3 mytest.py -v
test_list_equals (__main__.MyTests)
Checks whether two lists are equal ... ok
----------------------------------------------------------------------
Ran 1 test in 0.000s
OK
```

json reference

```
# load a json object as a dictionary from a file
json_dict = json.load(open("pizza.json"))

# pretty-print the json_dictionary with 4 spaces as indentation
# (useful for webscraping/debugging)
print(json.dumps(json_dict, indent=4))
```

requests reference

```
# send HTTP GET request to the specified URL,
# receive a Response object
response =
requests.get("https://en.wikipedia.org/api/rest_v1/page/summary/pizza")

# extract the json data (if it exists) as a
# dictionary from the response
response_json = response.json()
print(type(response_json))
<class 'dict'>
```

Takeaways

- Module usage and creation
- Script usage and creation
- How to write tests

CREDITS: This presentation template was created by **Slidesgo**, and includes icons by **Flaticon**, and infographics & images by **Freepik**